

Introduction

Every so often, LeetCode hands you a problem that looks like a throwaway puzzle about phone keypads but is secretly testing something much bigger: can you recognize when a greedy choice is actually optimal, not just convenient? **Minimum Number of Pushes to Type Word I** is exactly that kind of problem, and it's a favorite in interviews at companies like Amazon and Bloomberg because it rewards a specific kind of thinking — noticing that a problem's difficulty collapses once you stop looking at individual items and start looking at *positions*.

This is also a gentle, accessible introduction to a much larger idea in computer science: Huffman coding, the technique behind real-world file compression. You don't need to know anything about compression to solve this problem, but once you do solve it, you'll have already built the core intuition for it. That's what makes this a genuinely useful problem to master rather than a one-off trick question.

Problem Overview

You're given a word made entirely of unique lowercase letters — no letter repeats. You have a phone keypad with 8 keys, and you get to decide how to map letters onto those keys. Each key can hold multiple letters, arranged in an order you choose.

Here's the cost rule: the first letter assigned to a key costs 1 push to type it once. The second letter on that same key costs 2 pushes. The third costs 3, and so on. Your job is to arrange all the letters in the word across the 8 keys so that the total number of pushes needed to type the entire word is as small as possible.

Since the word has no duplicate letters, every letter appears in the word exactly once — you just need to type each one, one time, as cheaply as possible.

Example

Take the word `"xycdefghij"` — 10 distinct letters.

- Fill the first key group: 8 letters get placed as the "first letter" on 8 different keys, each costing 1 push.
- Only 2 letters are left. They become the "second letter" on two of those keys, each costing 2 pushes.

Total cost: $(8 \times 1) + (2 \times 2) = 8 + 4 = 12$.

Now try "abcdefghi" — 9 distinct letters.

- The first 8 letters each get their own key, costing 1 push each: $8 \times 1 = 8$.
- The 9th letter has nowhere cheap left to go — it becomes the second letter on one of the keys, costing 2 pushes.

Total cost: $8 + 2 = 10$.

Intuition

Picture a parking lot near a busy building entrance. The spots closest to the door are the most convenient — cars that come and go often should park there, not the ones that sit all day. That's the entire idea behind this problem, translated into typing costs instead of walking distance.

Because no letter repeats in the word, one crucial simplification kicks in immediately: every letter is "equally important" — there's no letter that shows up more than another, since each shows up exactly once. That means the *identity* of a letter is irrelevant to its cost. What matters is purely *where it lands* when you divide the letters into groups of 8.

Think of the cheap 1-push slots as scarce real estate. There are exactly 8 of them, no more. So the smartest thing you can do is fill all 8 of those before anyone is forced onto a more expensive slot. The 9th letter you place has no choice — the cheap slots are gone, so it lands in the 2-push group. The 17th letter would land in the 3-push group, and so on.

This gives you a clean rule: split the letters into consecutive groups of 8. The first group costs 1 push per letter, the second group costs 2 pushes per letter, the third costs 3, and so on. Total cost = sum of $(\text{group number}) \times (\text{letters in that group})$.

Note: for this specific version of the problem (word has unique letters), sorting by frequency doesn't actually change anything, since every letter has a frequency of exactly 1. The real algorithm shows its full power on the harder follow-up variant, "Minimum Number of Pushes to Type Word II," where letters *can* repeat and frequency sorting matters. But the bucket logic — the real "aha" — is identical in both.

Brute Force Solution

Idea: Try every possible way to assign letters to keys and positions, calculate the cost for each arrangement, and keep the minimum.

Algorithm: 1. Generate every permutation of letter-to-key-position assignments. 2. For each permutation, compute the total push cost. 3. Track the minimum cost seen.

Advantages: Guaranteed correct — it's an exhaustive search, so it can't miss the optimal answer.

Disadvantages: Explodes combinatorially. With just 26 letters, the number of arrangements reaches factorial-scale territory — computationally impossible to finish in any reasonable time.

Complexity: Time $O(n!)$ (or worse, depending on how positions are also permuted). Space grows just as fast if you track candidate arrangements.

This approach is only useful conceptually — it clarifies what "correct" means, but it's never something you'd actually run past a handful of letters.

Optimal Solution

The greedy bucket strategy, step by step:

1. **Count** how often each letter appears (in this problem, always 1, since letters are unique).
2. **Sort** the letters — by frequency, descending (a formality here, but essential in the repeated-letter variant).
3. **Bucket** the sorted letters into groups of 8.
4. For each letter at sorted position `i` (0-indexed), its **group number** is `i // 8 + 1`.
5. Its **cost contribution** is `group_number × frequency`.
6. **Sum** all contributions.

Position (0-indexed) Group number Cost per letter

0–7	1	1
8–15	2	2
16–23	3	3

This table is the entire algorithm. Everything else is bookkeeping.

Python Code

```
from collections import Counter

def minimum_pushes(word: str) -> int:
    # Count frequency of each letter (always 1 here, since letters are unique)
    freq = Counter(word)
```

```
# Sort frequencies from highest to lowest
counts = sorted(freq.values(), reverse=True)

total_pushes = 0
for index, count in enumerate(counts):
    group_number = index // 8 + 1
    total_pushes += group_number * count

return total_pushes
```

Code Walkthrough

- `Counter(word)` builds a dictionary mapping each letter to how many times it appears in the word.
- `freq.values()` pulls out just the counts, discarding which letter they belong to — we no longer need letter identity.
- `sorted(..., reverse=True)` orders those counts from most frequent to least frequent.
- The loop walks through each count with its index. `index // 8` finds which zero-indexed group of 8 the letter falls into; `+ 1` converts that into a 1-indexed push cost.
- `group_number * count` gives the total pushes contributed by all letters sharing that frequency; it accumulates into `total_pushes`.

Dry Run

Word: "abcdefghi"

1. `Counter` → each of `a` through `i` maps to `1`.
2. `sorted(freq.values(), reverse=True)` → `[1, 1, 1, 1, 1, 1, 1, 1, 1]` (9 ones).
3. Loop:
4. Indices 0–7 (8 letters): `group = 0//8+1 = 1` → each contributes `1 × 1 = 1`. Running total: `8`.
5. Index 8 (9th letter): `group = 8//8+1 = 2` → contributes `2 × 1 = 2`. Running total: `10`.
6. Return `10`. ✅ Matches the expected answer.

Complexity Analysis

- **Time:** $O(n \log n)$, dominated by the sort. Counting is $O(n)$, the final loop is $O(n)$. Sorting is the bottleneck, and it stays fast even as the word grows.
- **Space:** $O(n)$ for the `Counter` and the sorted list of counts.

This is correct because the bucket assignment is a direct, provable consequence of the greedy exchange argument: any arrangement that leaves a cheap slot empty while a letter pays a higher cost elsewhere can always be improved by swapping them — so the optimal arrangement always fills cheap slots first.

Alternative Solutions

You could skip explicit sorting and use a `heapq` to always pull the next letter in frequency order, but since letters here are unique (frequency always 1), this adds complexity without benefit. For the harder "Word II" variant with repeated letters, sorting is still simpler and just as fast as a heap-based approach, since you need the full ordering anyway.

Edge Cases

- **Single letter:** Word length 1 → answer is `1` (one push).
- **Fewer than 8 letters:** Every letter fits in the first group → answer equals word length.
- **Exactly 8 letters:** All cost 1 push each → answer equals 8.
- **9th letter (boundary crossing):** The moment word length exceeds a multiple of 8, cost jumps — this is the case most likely to trip people up.
- **Maximum length (26 unique lowercase letters):** Still handled cleanly; groups extend to a third bucket (17th–24th letters) and beyond.

Common Mistakes

1. Using 0-indexed group numbers instead of adding 1, undercounting every cost.
2. Forgetting to sort before bucketing (irrelevant here since frequencies are equal, but critical in the repeated-letter version).
3. Assuming key assignment order matters when letters are unique — it doesn't; only bucket size matters.
4. Off-by-one errors at bucket boundaries (letter 8 vs. letter 9).
5. Reaching for brute-force permutations out of habit instead of recognizing the greedy structure.
6. Forgetting the problem guarantees unique letters and re-adding unnecessary duplicate-handling logic.

Interview Questions

- How would this change if letters could repeat? (This is literally LeetCode's "Word II" follow-up.)

- Can you prove the greedy approach is optimal? (Exchange argument: swapping a cheaper letter into an expensive slot never increases cost when sorted by frequency.)
- What's the relationship between this problem and Huffman coding?
- How would you handle a keypad with a different number of keys, say 10 instead of 8?

Similar Problems

- **Minimum Number of Pushes to Type Word II** — the direct sequel, where letters repeat and frequency sorting actually matters.
- **Huffman Coding / Construct Huffman Tree** — the general-purpose ancestor of this bucket idea, used in real compression.
- **Task Scheduler (LeetCode 621)** — another frequency-count-and-greedy-arrange problem.
- **Reorganize String (LeetCode 767)** — greedy placement based on frequency to avoid adjacency conflicts.

Key Takeaways

The core lesson: when every item has equal weight, cost is decided by *position*, not identity. Counting, sorting, and bucketing into fixed-size groups is enough to beat any brute-force permutation search, and this exact pattern — fill the cheapest resource first — reappears constantly in interview problems and in real systems like data compression.

Watch the Video

Want to see this built up from scratch, mistake and all? Watch the full walkthrough here: {LINK}

About the Series

This problem is part of **Fun with Learning Technology**, a daily series that takes one LeetCode problem at a time and rebuilds the intuition behind it from zero — no assumed background, just the reasoning an experienced engineer actually uses to get to the answer.

Call To Action

If this clicked for you, tap like on the video, subscribe for tomorrow's problem, and consider subscribing to the Substack for the written breakdowns. Drop a comment with your own dry run, and

share this with anyone prepping for interviews.

Quick Review

Which pattern fits 'remap keys to minimize total presses'?

Greedy frequency assignment — when you see 'minimize total cost of assigning slots', sort by frequency and hand out cheapest slots first.

Time complexity of the keypad remapping solution?

$O(n \log n)$, dominated by sorting the 26 letters by frequency.

Space complexity of the frequency-count + sort approach?

$O(1)$ extra — fixed 26-letter count array, ignoring input storage.

Common bug when assigning letters to keypad positions?

Forgetting each key holds up to 3 letters, so position cost is $\text{index}/3+1$, not $\text{index}+1$.

Why does 'just assign in order' (no sorting) fail?

It ignores frequency, so a rare letter can land on a cheap key while a common one lands on an expensive one — wastes presses.

Interview follow-up: what if keys allowed 4 letters not 3?

Same greedy logic, just change divisor: $\text{cost} = \text{index}/4 + 1$ per letter, still sort by frequency descending.